



Sistemas Embebidos — Trabajo Práctico N° 3

Comunicación I2C y Display LCD en STM32F103

Integrantes:

Máximo Mariani — LU 1151848

Santiago Bolanca — LU 1176307

Docente: Dr. Ing. Leonardo J. Amet

Fecha de entrega: Junio de 2026

Índice

Índice	2
1. Introducción y objetivos	4
2. Hardware y conexionado	4
2.1 Tabla de conexiones	4
3. Etapa 1 — I2C scanner	5
3.1 Por qué un scanner antes del driver	5
3.2 Fundamento: bus open-drain y pull-ups	5
3.3 Configuración de I2C1	5
3.4 El scanner	6
3.5 Resultado obtenido	6
3.6 Diagnóstico real durante el armado: el pull-up faltante	8
3.7 Preguntas obligatorias de la Etapa 1	8
1. ¿Qué medís con un multímetro en SDA y SCL en reposo? ¿Qué valor esperás y por qué?	8
2. Si el scanner no encuentra nada, ¿las tres causas más probables y cómo las diferencías?	8
3. ¿Por qué el scanner prueba desde 0x01 y no desde 0x00? ¿Qué direcciones están reservadas?	8
4. Etapa 2 — Driver HD44780 por PCF8574	8
4.1 El mapeo PCF8574 → HD44780	8
4.2 Arquitectura del driver por capas	9
4.3 El pulso de EN: el corazón del driver	9
4.4 La secuencia de inicialización	10
4.5 Resultado obtenido	10
4.6 Nota de diseño: por qué uso i2c_transfer7()	11
4.7 Preguntas obligatorias de la Etapa 2	11
1. Explicar la secuencia de inicialización. ¿Por qué se envían tres veces 0x30 antes del 0x20?	11
2. ¿Cómo adaptarías el driver para un LCD de 20×4 en vez de 16×2?	11
3. Si el LCD muestra basura o bloques negros después de lcd_init(), ¿qué etapa falló y cómo lo diagnosticás?	11
5. Etapa 3 — Display dinámico	11
5.1 Fuente de datos e integración con el TP2	12
5.2 Base de tiempo: reutilizar TIM3 en lugar de agregar un timer	12

5.3 El patrón ISR levanta bandera, main loop actualiza.....	12
5.4 Actualización parcial sin parpadeo.....	13
5.5 Verificación de robustez	13
5.6 Preguntas obligatorias de la Etapa 3	13
1. ¿Por qué actualizar el LCD desde el lazo principal y no desde la ISR? ¿Qué pasaría si i2c_transfer7() se llamara dentro de una ISR?	13
2. Si el display muestra valores correctos al inicio pero después se congela o muestra basura, ¿tres hipótesis ordenadas por probabilidad?	14
6. Preguntas obligatorias de cierre	14
6.1 Manejo de NACK.....	14
6.2 Bus colgado y SDA/SCL trabados	14
6.3 Bug del flag BUSY del STM32F1.....	15
6.4 Pull-ups y velocidad del bus	15
7. Conclusiones	16
8. Anexo: organización de la evidencia	16

1. Introducción y objetivos

Este trabajo práctico lleva por primera vez el desarrollo fuera del propio microcontrolador. Hasta el TP2 toda la acción ocurría dentro del chip: se configuraban registros del ADC, del PWM o de los timers, y el silicio respondía. Aquí el desafío es distinto: se busca que la Blue Pill le hable a un integrado externo (el expansor PCF8574) a través de un bus de dos cables, y que ese integrado, a su vez, maneje un tercer dispositivo (el controlador HD44780 de un display LCD).

Esa cadena de tres eslabones es el corazón conceptual del TP. Cuando los datos viajan dentro del silicio, si algo no anda, el problema está en el código propio. Cuando viajan por dos cables que comparten varios chips, el problema puede estar en el código, el cableado, las resistencias pull-up, la velocidad del bus, el chip expansor o el controlador del LCD. Hay capas nuevas que diagnosticar, y por eso el trabajo se organizó en tres etapas, cada una aislando una capa de la pila de comunicación:

1. **Etapla 1 — I2C scanner:** verificar que la capa física y el protocolo I2C funcionan, detectando la dirección del esclavo en el bus.
2. **Etapla 2 — Driver HD44780:** mostrar texto estático, dominando la secuencia de inicialización del controlador y el mapeo PCF8574 → LCD.
3. **Etapla 3 — Display dinámico:** integrar el LCD con datos que cambian en tiempo real, reutilizando el ADC, el PWM y el botón del TP2, con el patrón profesional de interrupción que levanta bandera y lazo principal que actualiza.

Todo el desarrollo se realizó bare-metal sobre la Blue Pill, configurando los periféricos directamente con libopenm3, sin capa HAL.

2. Hardware y conexionado

La cadena física de comunicación es la siguiente: la Blue Pill habla I2C (dos cables, SCL y SDA) con el PCF8574; el PCF8574 es un expansor de entrada/salida que recibe un byte por I2C y lo refleja en sus 8 patas de salida; esas 8 patas están cableadas a las líneas de control y datos del HD44780, que es el controlador que finalmente maneja la pantalla LCD de 16×2.

El backpack que se compra comercialmente es justamente una placa con el PCF8574 ya soldado al LCD: uno ve un módulo I2C, pero adentro hay dos chips comunicándose en paralelo.

2.1 Tabla de conexiones

Señal	Pin Blue Pill	Conexión
SCL (I2C1)	PB6	SCL del backpack + pull-up 4.7 kΩ a 3.3 V
SDA (I2C1)	PB7	SDA del backpack + pull-up 4.7 kΩ a 3.3 V
UART TX	PA9	RX del adaptador USB-Serial (depuración)
ADC IN0	PA0	Cursor del potenciómetro de 10 kΩ
PWM (TIM2_CH2)	PA1	R 220 Ω → LED → GND
Botón (EXTI1)	PB1	Pulsador a 3.3 V + R 10 kΩ pull-down a GND

Alimentación	3.3 V / 5 V	VCC del backpack (lógica y pull-ups a 3.3 V)
Masa	GND	GND común a Blue Pill, backpack, LED y USB-Serial

La dirección del esclavo detectada por el scanner fue 0x27, lo que indica que el módulo usa el PCF8574 estándar (no la variante PCF8574A, que respondería en 0x3F).

3. Etapa 1 — I2C scanner

3.1 Por qué un scanner antes del driver

El desarrollo comienza por el scanner y no directamente por el LCD por una razón de método: el scanner es un test de aislamiento de la capa física más el protocolo I2C. Si encuentra una dirección, se confirma con certeza que los pull-ups están bien, que el cableado está bien, que el periférico I2C1 está bien configurado y que el esclavo responde. Recién con esa base confirmada tiene sentido atacar la secuencia del HD44780. Si se arrancara directo con el driver del LCD y no funcionara, no podría distinguirse si el problema es del bus, del expansor o de la inicialización del controlador.

3.2 Fundamento: bus open-drain y pull-ups

El concepto que sostiene todo I2C es que es un bus open-drain. A diferencia de una salida push-pull, donde el chip puede tanto forzar el pin a 1 (conectándolo a Vcc) como a 0 (conectándolo a GND), en open-drain el chip solo puede tirar la línea a 0; hacia 1 no la puede llevar, lo único que puede hacer es soltarla (alta impedancia). Esto es necesario porque en el bus hay varios chips compartiendo las mismas dos líneas: si dos intentaran forzar la línea en sentidos opuestos se cortocircuitarían.

La resistencia pull-up es la que pone la línea en 1 cuando nadie la baja: una resistencia entre la línea y 3.3 V que lleva el bus a nivel alto cuando todos los chips sueltan. Cuando un chip quiere mandar un 0, conecta la línea a GND y vence al pull-up. De acá se derivan tres consecuencias prácticas: sin pull-ups la línea nunca llega a 1 y el bus queda muerto; cualquier dispositivo puede colgar el bus tirando SDA a 0 y no soltándola; y el valor del pull-up es un compromiso entre velocidad de los flancos y corriente de drenaje (desarrollado en la pregunta de cierre 4).

3.3 Configuración de I2C1

El periférico I2C1 cuelga del bus APB1, que en la configuración de reloj utilizada (HSE 8 MHz × PLL 9 = 72 MHz, dividido por 2) corre a 36 MHz. Esa frecuencia es la referencia que necesita la función de configuración de velocidad para generar correctamente la temporización de SCL. Los pasos de inicialización son:

```
static void i2c1_setup(void)
{
    gpio_set_mode(GPIOB, GPIO_MODE_OUTPUT_50_MHZ,
                  GPIO_CNF_OUTPUT_ALTFN_OPENDRAIN,
                  GPIO6 | GPIO7); /* PB6=SCL, PB7=SDA */
}
```

```
i2c_peripheral_disable(I2C1);          /* requisito del F1 */
i2c_set_speed(I2C1, i2c_speed_sm_100k, /* 100 kHz standard */
              APB1_FREQ_MHZ);          /* 36 MHz */
i2c_peripheral_enable(I2C1);
}
```

Los pines PB6 y PB7 se configuran como función alternativa open-drain: alternativa porque el control lo toma el periférico I2C, no el GPIO; open-drain porque el bus I2C lo exige.

3.4 El scanner

El scanner recorre las direcciones de 7 bits válidas e intenta una transferencia de cero bytes en cada una. El protocolo avisa solo si hay alguien: el esclavo confirma su existencia con el ACK (un 0 lógico en SDA durante el noveno pulso de clock). Si `i2c_transfer7()` devuelve un valor no negativo, hubo ACK y hay un dispositivo en esa dirección.

```
for (uint8_t addr = 0x01; addr <= 0x7E; addr++) {
    int ret = i2c_transfer7(I2C1, addr, &dummy, 0, NULL, 0);
    if (ret >= 0) {
        /* ACK: hay dispositivo en esta direccion */
        snprintf(buf, sizeof(buf),
                 " Dispositivo I2C en 0x%02X\r\n", addr);
        usart_send_str(buf);
        found++;
    }
}
```

3.5 Resultado obtenido

La salida por UART confirmó un único dispositivo en la dirección esperada:

```
maximomariani@PC-DANIEL:~/BLUEPILL$ picocom -b 115200 /dev/ttyUSB0
Type [C-a] [C-h] to see available commands
Terminal ready
--- Inicio de barrido ---
Dispositivo I2C en 0x27
Total: 1 dispositivo(s) encontrado(s).
--- Fin de barrido ---
```

Que aparezca un solo dispositivo es lo correcto (un backpack equivale a un PCF8574). El 0x27 confirma además que es el PCF8574 estándar.

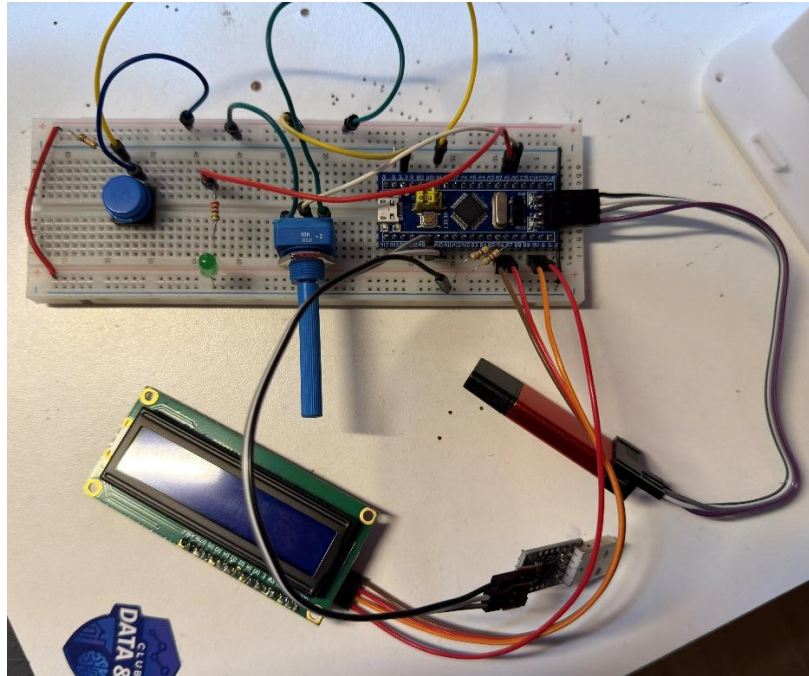


Figura 1. Conexión de la Etapa 1: Blue Pill, backpack PCF8574 y resistencias pull-up de 4.7 k Ω en SDA y SCL.

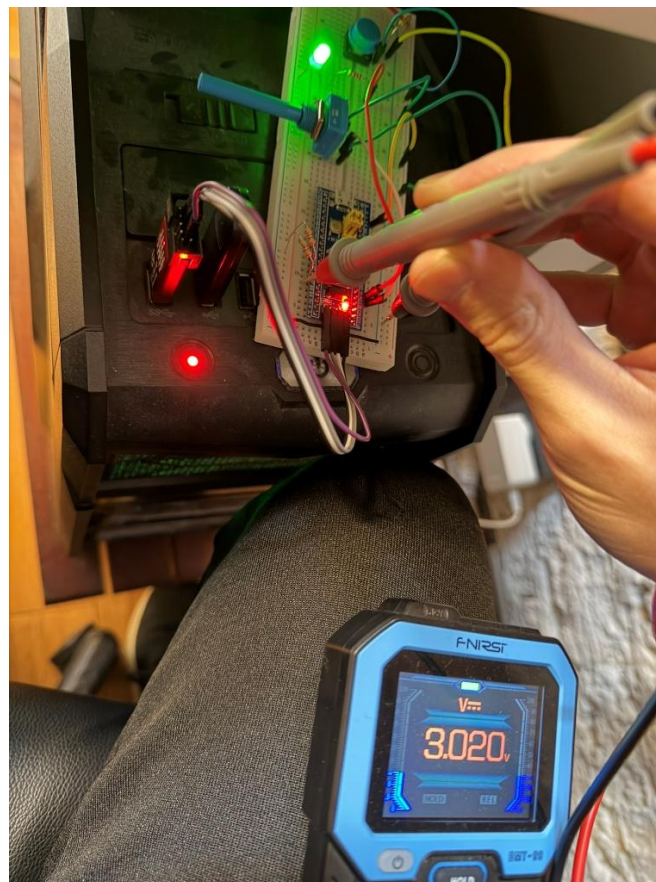


Figura 2. Medición con multímetro de SDA y SCL en reposo: ~3.3 V en ambas líneas, confirmando el funcionamiento de los pull-ups.

3.6 Diagnóstico real durante el armado: el pull-up faltante

Vale la pena documentar un problema surgido durante el armado, porque refleja directamente la causa más común de fallo en esta etapa. Antes de flashear el código del scanner medimos PB6 y PB7 respecto a GND y obtuvimos alrededor de 1 V en ambos, no los 3.3 V esperados. El motivo era simple: todavía no se habían colocado las resistencias pull-up de 4.7 kΩ. Sin ellas, las líneas quedaban flotando y el multímetro leía una tensión fantasma producto de ruido y fugas. Al colocar las dos resistencias, la medición pasó a 3.3 V limpios en ambas líneas. Esto confirma en la práctica que la medición con multímetro en reposo es el primer test de diagnóstico antes de tocar una sola línea de software.

3.7 Preguntas obligatorias de la Etapa 1

1. ¿Qué medís con un multímetro en SDA y SCL en reposo? ¿Qué valor esperás y por qué?

Se esperan aproximadamente 3.3 V en ambas líneas. En reposo ningún dispositivo está tirando las líneas a 0, y como el bus es open-drain, las resistencias pull-up de 4.7 kΩ llevan ambas líneas al nivel de alimentación. Si midiera 0 V, faltaría el pull-up o habría un corto a GND; si se midiera un valor intermedio e inestable (como el 1 V obtenido antes de poner las resistencias), las líneas estarían flotando sin pull-up. Esta medición se verificó en el propio armado.

2. Si el scanner no encuentra nada, ¿las tres causas más probables y cómo las diferenciás?

- **Pull-ups faltantes o mal conectados (la más común):** se diagnostica con el multímetro. Si en reposo no hay ~3.3 V en las líneas, es esto. Fue exactamente el caso ocurrido durante el armado.
- **Cableado SDA/SCL invertido o suelto:** se diagnostica revisando físicamente o intercambiando PB6 y PB7. Con el multímetro las líneas pueden estar bien a 3.3 V, pero el micro no logra comunicarse.
- **Esclavo sin alimentación o fuera de rango de direcciones:** se verifica que VCC del backpack tenga tensión. Con osciloscopio se vería que el micro genera START y los pulsos de clock, pero nadie responde con ACK.

3. ¿Por qué el scanner prueba desde 0x01 y no desde 0x00? ¿Qué direcciones están reservadas?

La dirección 0x00 es la de general call (broadcast), reservada por el estándar. Además, 0x01–0x07 están reservadas para usos especiales (start byte, CBUS, etc.) y 0x78–0x7F para direccionamiento de 10 bits. El rango útil de esclavos de 7 bits es 0x08–0x77, así que barrer de 0x01 a 0x7E es una convención segura que cubre todo el espacio práctico sin tocar la dirección de broadcast ni el tope reservado.

4. Etapa 2 — Driver HD44780 por PCF8574

4.1 El mapeo PCF8574 → HD44780

Las 8 salidas del PCF8574 están cableadas al HD44780 según el mapeo estándar de los backpacks:

Bit PCF8574	Pin HD44780	Función
P0	RS	Register Select (0 = comando, 1 = dato)
P1	RW	Read/Write (a GND en el backpack, siempre 0)
P2	EN	Enable (el flanco descendente lacha el dato)
P3	BL	Backlight (1 = encendido)
P4	D4	Bit de datos 4 (nibble superior)
P5	D5	Bit de datos 5
P6	D6	Bit de datos 6
P7	D7	Bit de datos 7

El detalle crucial es que solo hay cuatro líneas de datos (D4–D7), no ocho. Esto obliga a trabajar en modo 4 bits: cada byte que se envía al LCD hay que partirlo en dos nibbles (medios bytes de 4 bits) y enviarlos uno tras otro, primero el nibble alto y después el bajo.

4.2 Arquitectura del driver por capas

El driver se organizó en capas, de la más baja a la más alta, lo que mantiene cada función con una única responsabilidad:

- `pcf8574_write(byte)`: una sola escritura I2C cruda al expansor. Le suma el flag de backlight para no apagar la luz entre transferencias. Todo lo que va al LCD pasa por acá.
- `lcd_send_nibble(nibble, modo)`: coloca el nibble en los 4 bits altos del byte del PCF8574, agrega los bits de control y genera el pulso de EN escribiendo dos veces (EN=1 y luego EN=0).
- `lcd_send_byte(valor, modo)`: parte el byte en nibble alto y bajo y los envía en orden. Solo es válido una vez que el LCD ya está en modo 4 bits.
- **API pública** (`lcd_init`, `lcd_clear`, `lcd_set_cursor`, `lcd_write_char`, `lcd_write_string`, `lcd_print_line`): lo que se utiliza desde el main.

4.3 El pulso de EN: el corazón del driver

El HD44780 no lee los datos continuamente: los captura solo en el instante en que la línea EN pasa de 1 a 0 (flanco descendente). Funciona como un enter: primero se colocan los datos en las líneas y después se da el pulso de EN para indicar que deben leerse en ese instante. Por eso enviar un nibble implica escribir el mismo byte dos veces, una con EN=1 (datos presentes) y otra con EN=0 (flanco de bajada que lacha). Y como cada byte real son dos nibbles, mandar un byte al LCD implica cuatro escrituras I2C al PCF8574.

```
static void lcd_send_nibble(uint8_t nibble, uint8_t mode)
{
    uint8_t base = (uint8_t)((nibble & 0x0F) << 4);
    if (mode == LCD_DATA) base |= LCD_RS;
```

```
pcf8574_write(base | LCD_EN); /* EN=1: datos presentes */
pcf8574_write(base);          /* EN=0: flanco -> latch */
}
```

4.4 La secuencia de inicialización

Esta es la parte más delicada del driver. Al encender, el HD44780 no sabe si se le hablará en 4 u 8 bits: arranca asumiendo 8 bits. Para llevarlo a 4 bits hay una secuencia ritual definida en el datasheet que hay que respetar al pie de la letra:

```
void lcd_init(void) {
    delay_ms(50);                /* estabilizacion >40 ms */
    lcd_send_nibble(0x03, LCD_CMD); /* fuerza modo 8-bit (1) */
    delay_ms(5);
    lcd_send_nibble(0x03, LCD_CMD); /* fuerza modo 8-bit (2) */
    delay_ms(1);
    lcd_send_nibble(0x03, LCD_CMD); /* fuerza modo 8-bit (3) */
    delay_ms(1);
    lcd_send_nibble(0x02, LCD_CMD); /* pasa a modo 4-bit */
    delay_ms(1);
    lcd_send_byte(0x28, LCD_CMD); /* 4-bit, 2 lineas, 5x8 */
    lcd_send_byte(0x08, LCD_CMD); /* display OFF */
    lcd_send_byte(0x01, LCD_CMD); /* clear (>1.5 ms) */ delay_ms(2);
    lcd_send_byte(0x06, LCD_CMD); /* entry mode: cursor a la derecha */
    lcd_send_byte(0x0C, LCD_CMD); /* display ON, cursor OFF */
}
```

Los primeros envíos van como nibble único (no como byte partido) porque el chip todavía está en modo 8 bits durante la inicialización: interpreta un solo envío de 4 bits por D4–D7 como un comando completo. Recién después del 0x20 entra en modo 4 bits y empieza a esperar dos nibbles por byte. Por eso no se puede usar la función genérica de mandar byte durante el arranque.

4.5 Resultado obtenido

Tras la inicialización, el LCD muestra texto estático estable y nítido en las dos líneas (I2C LCD OK / TP3 Embebidos), confirmando que toda la cadena init + mapeo + pulsos de EN funciona correctamente.

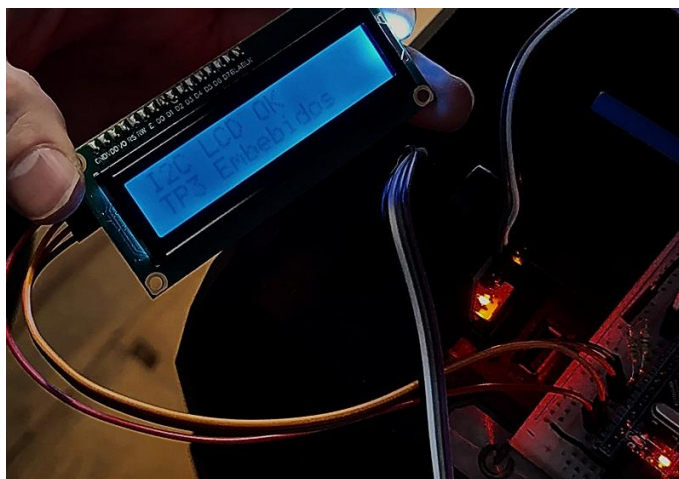


Figura 3. LCD mostrando texto estático legible tras `lcd_init()` (Etapa 2).

4.6 Nota de diseño: por qué uso `i2c_transfer7()`

Para la escritura I2C de bajo nivel se utiliza `i2c_transfer7()` de `libopencm3`, que encapsula la máquina de estados del periférico (generación de START, envío de dirección, espera de los flags de transmisión y generación de STOP) y además maneja internamente la errata de lectura de dos bytes del STM32F1. La decisión prioriza robustez y legibilidad: la librería resuelve los casos de borde del protocolo de forma probada, y el razonamiento sobre el bajo nivel del bus queda reflejado en las respuestas a las preguntas de cierre, donde se analiza el manejo de NACK, la recuperación de un bus colgado y el bug del flag BUSY.

4.7 Preguntas obligatorias de la Etapa 2

1. Explicar la secuencia de inicialización. ¿Por qué se envían tres veces 0x30 antes del 0x20?

Al encender, el HD44780 puede quedar en un estado indeterminado: en modo 8 bits, o a mitad de recibir un byte de 4 bits de una sesión anterior. Los tres envíos de 0x30 (modo 8 bits) garantizan que, sin importar en qué estado quedó, el chip termine en un modo 8 bits conocido. Si el chip estaba esperando el segundo nibble de un byte, el primer 0x30 completa ese byte; el segundo y el tercero lo dejan definitivamente sincronizado en 8 bits. Recién entonces el 0x20 lo pasa de forma confiable a 4 bits. Las esperas entre envíos (5 ms tras el primero, ~100 µs tras los siguientes) respetan los tiempos mínimos que el controlador necesita para procesar cada comando.

2. ¿Cómo adaptarías el driver para un LCD de 20×4 en vez de 16×2?

El protocolo de bajo nivel (nibble, byte, pulso de EN, init) no cambia: sigue siendo un HD44780 en modo 4 bits. Lo que cambia es el direccionamiento DDRAM y el ancho de línea. En un 20×4, las cuatro filas no son consecutivas en memoria: las direcciones base son 0x00 (fila 0), 0x40 (fila 1), 0x14 (fila 2) y 0x54 (fila 3). Se adaptaría `lcd_set_cursor` para usar una tabla de cuatro offsets en lugar de dos, y `lcd_print_line` para rellenar a 20 caracteres en vez de 16. El resto del driver queda intacto.

3. Si el LCD muestra basura o bloques negros después de `lcd_init()`, ¿qué etapa falló y cómo lo diagnosticás?

Una fila de bloques negros suele significar que el LCD tiene alimentación pero no recibió la secuencia de init correctamente, o que el contraste está mal ajustado. El primer paso es girar el trimmer de contraste del backpack, porque a veces es solo eso. Si el problema persiste, la sospecha recae en la inicialización: si el backlight enciende pero no hay texto, la comunicación I2C llega (por eso enciende la luz) pero la init falló, típicamente por timing insuficiente entre comandos o por un error en el orden de los nibbles. El diagnóstico consistiría en verificar que el scanner sigue viendo 0x27 (descarta problema de bus) y luego revisar con osciloscopio o aumentar los delays de `lcd_init` para descartar un problema de tiempos, frecuente en los clones más baratos.

5. Etapa 3 — Display dinámico

5.1 Fuente de datos e integración con el TP2

Se optó por la integración más completa: se reutilizó el sistema entero del TP2 (ADC en PA0, PWM del LED en PA1, botón con anti-rebote en PB1/EXTI1 y TIM3 como base de tiempo) y se le agregó el LCD. El botón cicla tres modos que cambian simultáneamente el brillo del LED y lo que muestra el display:

- **Modo 0 (DIRECTO):** el LED brilla proporcional al potenciómetro; el LCD muestra el valor del ADC en crudo y su equivalente en mV.
- **Modo 1 (INVERSO):** el LED brilla inverso al potenciómetro; el LCD muestra el duty del PWM en porcentaje.
- **Modo 2 (FIJO 50):** el LED queda al 50 %; el LCD muestra un texto fijo indicándolo.

5.2 Base de tiempo: reutilizar TIM3 en lugar de agregar un timer

TIM3 ya disparaba cada 50 ms en el TP2 como base temporal del muestreo del ADC y del anti-rebote. En lugar de configurar un TIM4 nuevo para el LCD, se agregó un divisor por software dentro de la ISR de TIM3: cada dos disparos ($2 \times 50 \text{ ms} = 100 \text{ ms} = 10 \text{ Hz}$) se levanta la bandera de refresco del display. Así el ADC se muestrea a 20 Hz y el LCD se refresca a 10 Hz, todo con una única base de tiempo.

```
void tim3_isr(void) {
    if (timer_get_flag(TIM3, TIM_SR_UIF)) {
        timer_clear_flag(TIM3, TIM_SR_UIF);
        /* muestreo ADC -> levanta new_data */
        adc_value = adc_read_regular(ADC1);
        new_data = true;
        /* divisor 50->100 ms para el LCD */
        if (++lcd_div >= LCD_REFRESH_TICKS) {
            lcd_div = 0;
            tick_lcd = true; /* solo levanta bandera */
        }
        /* ... anti-rebote del boton ... */
    }
}
```

5.3 El patrón ISR levanta bandera, main loop actualiza

El punto central de la etapa es que la ISR nunca toca el LCD. La ISR solo lee el ADC y levanta banderas (new_data, tick_lcd), y devuelve el control en microsegundos. El lazo principal, al ver las banderas, hace el trabajo: el cálculo del duty y la actualización del PWM (rápido, un registro) y la actualización del LCD por I2C (lento, decenas de escrituras). La transferencia I2C bloqueante vive en el main, no en la interrupción.

```
while (1) {
    if (new_data) { /* trabajo rapido */
        new_data = false;
        duty = calculate_duty(adc_value, mode);
        timer_set_oc_value(TIM2, TIM_OC2, duty);
    }
    if (tick_lcd) { /* trabajo lento: I2C */
        tick_lcd = false;
        refresh_lcd(adc_value, mode, duty);
    }
}
```

```
}
```

5.4 Actualización parcial sin parpadeo

Para evitar el parpadeo no se usa `lcd_clear` en cada refresco (eso borraría y rellenaría la pantalla 10 veces por segundo). En su lugar, la función `lcd_print_line` reposiciona el cursor al inicio de la fila y sobrescribe los 16 caracteres, rellenando con espacios donde la cadena nueva es más corta. Así los dígitos viejos se borran sin limpiar toda la pantalla. Los buffers se dimensionan a 17 bytes (16 caracteres + terminador) y se formatean con `sprintf`, que corta si se excede, garantizando que nunca se desborden los 16 caracteres de la línea.

5.5 Verificación de robustez

El sistema se dejó funcionando varios minutos seguidos. El display sigue el potenciómetro en tiempo real sin parpadeo ni corrupción, responde a los cambios de modo del botón (reflejados a la vez en el LED y en la pantalla) y no muestra basura. El refresco a 10 Hz se percibe fluido y deja el bus holgado.

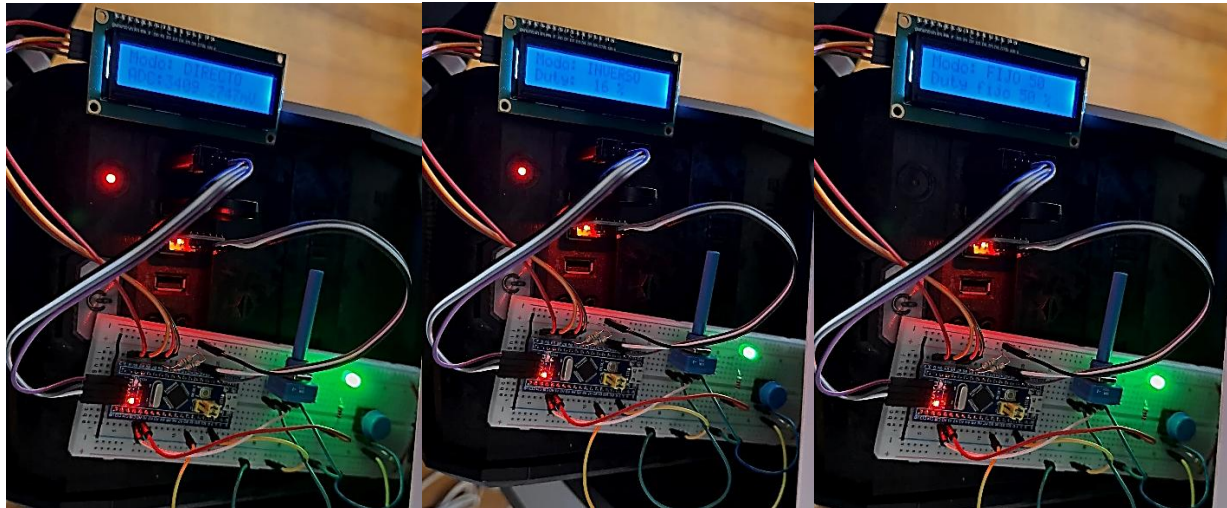


Figura 4. Secuencia del display actualizándose en tiempo real y ciclando entre modos con el botón (Etapa 3).

5.6 Preguntas obligatorias de la Etapa 3

1. ¿Por qué actualizar el LCD desde el lazo principal y no desde la ISR? ¿Qué pasaría si `i2c_transfer7()` se llamara dentro de una ISR?

Actualizar el LCD implica varias transferencias I2C, y cada `i2c_transfer7()` es bloqueante: espera el ACK del esclavo y la transmisión de cada byte. Mostrar un número de varios dígitos son decenas de escrituras, lo que a 100 kHz toma del orden de milisegundos. Si todo eso ocurriera dentro de la ISR, la interrupción pasaría de durar microsegundos a durar milisegundos, y durante ese tiempo bloquearía las demás interrupciones de igual o menor prioridad (el muestreo del ADC, el botón). Peor todavía: si la transferencia se colgara esperando un ACK que no llega, el sistema quedaría trabado dentro de la ISR de forma permanente. La regla es que las ISR deben ser cortas y no bloqueantes; el trabajo pesado va al lazo principal.

2. Si el display muestra valores correctos al inicio pero después se congela o muestra basura, ¿tres hipótesis ordenadas por probabilidad?

- **Bus I2C colgado por ruido o un glitch (más probable):** un transitorio dejó al esclavo o al periférico en un estado inconsistente, o el flag BUSY del STM32F1 se trabó. El diagnóstico consistiría en detectar el timeout en las transferencias y aplicar la secuencia de recuperación de la pregunta de cierre 2.
- **Desbordamiento del buffer de sprintf:** si un valor inesperado produjera una cadena más larga que el buffer, se corrompería la memoria adyacente. Se descartaría revisando los tamaños de buffer y los formatos.
- **Problema de alimentación o conexión intermitente:** un cable flojo o una caída de tensión bajo carga. El diagnóstico consistiría en medir la alimentación del backpack mientras ocurre la falla y revisar físicamente las conexiones.

6. Preguntas obligatorias de cierre

6.1 Manejo de NACK

Si `i2c_transfer7()` devuelve error (NACK) durante una escritura al LCD, el firmware actual ignora el retorno en la ruta del driver, porque en condiciones normales el esclavo siempre responde. Para hacerlo robusto, se implementarían reintentos: la escritura se envolvería en un bucle que reintenta la transferencia hasta un máximo, con una pequeña espera entre intentos. Un número razonable es de 3 a 5 reintentos antes de declarar el bus como fallado, porque si tras varios intentos consecutivos el esclavo sigue sin responder, es muy improbable que sea un glitch transitorio y muy probable que sea un problema persistente (cable suelto, esclavo colgado). Al declarar el bus fallado, se encendería una señalización de error (por ejemplo el LED o un mensaje por UART) y se aplicaría la secuencia de recuperación del punto siguiente.

```
static int lcd_write_retry(uint8_t b) {
    for (int i = 0; i < 5; i++) {
        if (i2c_transfer7(I2C1, ADDR, &b, 1, NULL, 0) >= 0)
            return 0;          /* éxito */
        delay_ms(1);
    }
    return -1;                 /* bus fallado tras 5 intentos */
}
```

6.2 Bus colgado y SDA/SCL trabados

Si un esclavo se cuelga a mitad de transacción y deja SDA en nivel bajo, el bus queda inutilizable porque el maestro no puede generar una condición de START válida. La recuperación por software, sin reiniciar el micro, consiste en forzar manualmente pulsos de clock para que el esclavo termine de transmitir el byte que quedó a medias y libere SDA:

1. Reconfigurar PB6 (SCL) y PB7 (SDA) como GPIO de salida open-drain comunes, sacándolos del control del periférico I2C.
2. Generar hasta 9 pulsos de clock manuales en SCL, verificando entre cada uno si SDA se liberó (volvió a nivel alto). Nueve pulsos garantizan que el esclavo complete cualquier byte que estuviera transmitiendo.

3. Generar manualmente una condición de STOP (SDA pasa de 0 a 1 con SCL en alto) para dejar el bus en reposo limpio.
4. Reconfigurar los pines como función alternativa I2C y resetear el periférico (ver punto 6.3) antes de reanudar las transferencias.

6.3 Bug del flag BUSY del STM32F1

El periférico I2C del STM32F103 tiene un bug conocido por el cual el flag BUSY puede quedar trabado en alto tras ciertas condiciones (por ejemplo, un glitch en las líneas durante el arranque), impidiendo iniciar nuevas transferencias. Para prevenirlo, en la inicialización se aplica un pulso de reset del periférico antes de configurarlo:

```
static void i2c1_setup(void) {
    rcc_periph_reset_pulse(RST_I2C1); /* reset: limpia BUSY */
    i2c_peripheral_disable(I2C1);
    i2c_set_speed(I2C1, i2c_speed_sm_100k, APB1_FREQ_MHZ);
    i2c_peripheral_enable(I2C1);
}
```

El pulso de reset deja el periférico en su estado por defecto, con el flag BUSY en cero, sin importar en qué estado hubiera quedado de una ejecución anterior. En runtime, se detectaría que el flag se trabó si una transferencia devuelve timeout mientras el bit BUSY permanece en alto pese a que las líneas físicas (medidas o leídas como GPIO) están en reposo a nivel alto. La recuperación consiste en aplicar la secuencia de liberación del bus del punto 6.2 y luego repetir el pulso de reset del periférico, que es la forma más segura de limpiar el BUSY trabado sin reiniciar todo el micro.

6.4 Pull-ups y velocidad del bus

El valor del pull-up determina la velocidad con que las líneas suben de 0 a 1. Las líneas tienen una capacitancia parásita (del cable y de las entradas de los chips); el pull-up y esa capacitancia forman un circuito RC, y el tiempo de subida es proporcional al producto $R \times C$. Un pull-up más chico (menor R) sube las líneas más rápido, permitiendo mayor velocidad de bus, pero exige más corriente al chip que tira la línea a 0.

- **Con 1 kΩ:** los flancos de subida son muy rápidos (bueno para alta velocidad, como Fast Mode a 400 kHz), pero la corriente que debe drenar el transistor de salida para mantener un 0 lógico válido es alta (más de 3 mA a 3.3 V). Eso puede acercarse o superar el límite del pin y, en el peor caso, no llegar a un nivel bajo válido.
- **Con 10 kΩ:** la corriente de drenaje es baja y cómoda, pero los flancos de subida son lentos. A 400 kHz la línea podría no alcanzar el nivel alto antes del siguiente flanco de clock, corrompiendo la comunicación. A 100 kHz suele funcionar, pero con menos margen.

El valor de 4.7 kΩ es el compromiso estándar para 100 kHz (Standard Mode): flancos suficientemente rápidos y corriente de drenaje moderada. Por eso es el valor recomendado y el que se utilizó.

7. Conclusiones

El TP3 cerró el salto desde los periféricos internos hacia la comunicación con dispositivos externos. La organización en tres etapas (scanner, driver estático, display dinámico) resultó clave para diagnosticar por capas: cada etapa confirmó una porción de la pila de comunicación antes de construir la siguiente encima.

El aprendizaje más concreto fue entender el bus I2C como un sistema open-drain donde los pull-ups no son un accesorio sino una condición de funcionamiento, algo que comprobamos en la práctica cuando el scanner no podía funcionar por las resistencias faltantes. En el driver, el concepto central fue el modo 4 bits y el rol del flanco descendente de EN como mecanismo de latcheo. Y en la integración, el patrón de interrupción que levanta bandera y lazo principal que actualiza demostró por qué las operaciones bloqueantes como las transferencias I2C no pueden vivir dentro de una ISR.

El sistema final integra cinco periféricos coordinados (ADC, PWM, EXTI, TIM3 y I2C) sobre una única base de tiempo, con el display reflejando en tiempo real el estado del sistema y respondiendo a las entradas del usuario, todo implementado bare-metal a nivel de registro mediante libopencm3.

8. Anexo: organización de la evidencia

Toda la evidencia técnica referida en el informe está organizada en el repositorio del trabajo, con la siguiente estructura:

proyecto/ contiene el código fuente y el Makefile necesarios para compilar el firmware. informe/ incluye este documento en formato .docx y .pdf. evidencia/ agrupa las capturas y materiales de respaldo, divididos en etapas correspondientes a las fases de desarrollo (I2C scanner, driver HD44780 por PCF8574, e integración del display dinámico con los datos del TP2); build/ contiene la salida de arm-none-eabi-size y los artefactos de compilación; y conexionado/ la fotografía del protoboard armado.

Repositorio en GitHub: <https://github.com/maxmariani01/tp3-embebidos-mariani>
